Содержание

Вступление 3
Теоретическое введение..... 4
Симуляция Раби импульсов 10
Обучение нейронной сети 15
Заключение..... 18
Список литературы 19
Приложение..... 20

Вступление

Квантовые вычисления – это перспективная область современных разработок и исследований. В отличие от “классических” компьютеров, где в качестве элементарных единиц информации используются биты, принимающие значения 0 или 1, в квантовых компьютерах используются квантовые биты – физические системы, находящиеся в квантовой суперпозиции состояний, которые соответствуют 0 или 1. Это позволяет, в теории, производить на квантовых компьютерах вычисления, которые являются недостижимыми для любых современных классических компьютеров, для некоторых классов задач.

Важнейшим аспектом квантовых вычислений является квантовое управление – область квантовой физики и квантовой информатики, которая занимается управлением динамикой квантовых систем с помощью внешних воздействий (например, лазерного излучения, магнитных полей или электрических импульсов) с целью реализации заданных квантовых операций или состояний.

Целью данной работы является, теоретическое описание явления Раби осцилляций – изменения квантового состояния системы, под действием электромагнитного поля. Далее задача состоит в том, чтобы с использованием языка программирования python, написать программу способную моделировать действие Раби импульса на простейшую модель квантового бита (не учитывающую тепловые эффекты, декогеренцию, диссипацию и т.д.).

На финальном этапе работы требуется написать нейросеть, которая способна принимать на вход произвольный унитарный оператор, соответствующий произвольному однокубитному вентилю и выводить параметры прямоугольного Раби импульса, длительностью в одну единицу времени (частота, фаза, имплитуда).

Теоретическое введение

Квантовый бит (Кубит) – это квантовая система, которая может находиться в суперпозиции двух базисных состояний. Состояние кубита описывается волновой функцией:

$$|\psi\rangle = \alpha\,|0\rangle + \beta|1\rangle \qquad (1)$$
$$|\alpha\,|^2 + |\beta|^2 = 1$$

Здесь $|0\rangle$ и $|1\rangle$ – два линейно независимых состояния системы, между которыми можно реализовать энергетические переходы. Часто выбираются состояния, соответствующие двум нижним уровням энергии системы, условно, нулевому и первому. В реальности у квантовой системы могут быть более высокие энергетические уровни $|2\rangle, |3\rangle$ и т. д. Но при описании кубита мы исходим из приближения, в котором вероятность нахождения системы, в котором вероятность перехода на эти уровни пренебрежимо мала. Чтобы такое приближение было справедливым, энергетическая щель между уровнем $|1\rangle$ и $|2\rangle$ должна быть много больше, чем между $|0\rangle$ и $|1\rangle$.

Примерами физических реализаций таких систем являются сверхпроводящие контуры с Джозефсоновскими переходами, квантовые точки, ионные ловушки и так далее.

Состояние кубита можно визуально представить в виде вектора, исходящего из начала координат и указывающего на точку на единичной сфере. Для этого необходимо параметризовать комплексные коэффициенты α и β через углы φ и θ следующим образом:

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}|1\rangle \qquad (2)$$

Здесь угол φ определяет разность фаз между базисными состояниями $|0\rangle$ и $|1\rangle$ соответствует азимутальному углу в сферической системе координат (углом между проекцией вектора на XY и положительным направлением оси X). Угол θ определяет абсолютные значения коэффициентов α и β, и задаёт

полярный угол в этой системе координат (угол между вектором и положительным направлением оси Z). Таким образом, каждому чистому состоянию кубита соответствует точка на сфере – сфере Блоха. Сфера Блоха изображена на рисунке 1.

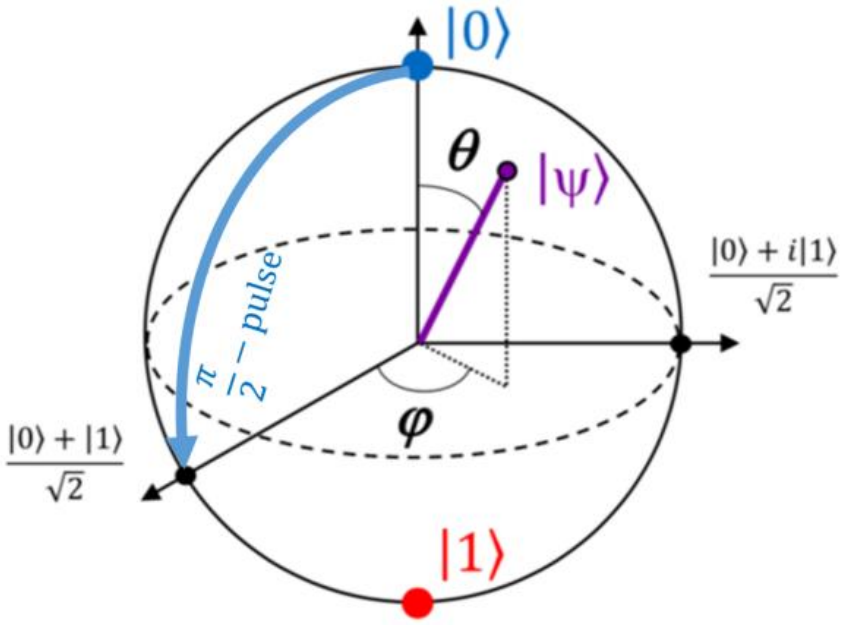


Рисунок 1. Сфера блоха

Для управления состоянием одного кубита и изменения его положения на сфере Блоха используются однокубитные вентили. Однокубитный вентиль — это унитарный оператор на пространстве состояний одного кубита, задаваемый матрицей 2×2. Произвольный оператор в пространстве сферы Блоха можно выразить как поворот на некий угол θ, вокруг оси $\vec{n}$ следующим образом: [1]

$$U = e^{-i\frac{\theta}{2}(\vec{\sigma}\cdot\vec{n})} \tag{3}$$

где $\vec{\sigma} = (\sigma_x,\ \sigma_y,\ \sigma_z)$ – вектор Паули

Для практической реализации однокубитных вентилей большое значение имеет эффект периодического перераспределения амлитуд

вероятностей между энергетическими уровнями системы под действием электромагнитного поля. Эти колебания называются Раби-осцилляциями.

Рассмотрим гамильтониан двухуровневой квантовой системы в классическом электромагнитном поле. Для этого воспользуемся дипольным приближением – предположением о том, что размер кубита много меньше длины волны электромагнитного поля. В таком случае векторный электромагнитный потенциал:

$$A(r,t) \approx A(t),\ \nabla A \approx 0 \tag{4}$$

Оператор Гамильтона в электромагнитном поле[2]:

$$\widehat{H} = \frac{(\hat{p} - e\vec{A}(t))^2}{2m} + e\varphi(\vec{r},t) + V(\vec{r}) \tag{5}$$

Используем дипольную калибровку:

$$\chi = -(\hat{r} \cdot \vec{A}(t)) \tag{6}$$

$$\vec{A}'(t) = \vec{A}(t) + \nabla\chi = 0 \tag{7}$$

$$\varphi'(\vec{r},t) = \varphi(\vec{r},t) - \frac{1}{c}\frac{\partial\chi}{\partial \mathrm{t}} = -\left(\hat{r} \cdot \vec{E}(t)\right) \tag{8}$$

Подставляя (7) и (8) в (5), в матричном представлении, в базисе собственных состояний гамильтониан запишется следующим образом:

$$\widehat{H} = \widehat{H}_0 - (\hat{d} \cdot \vec{E}) \tag{9}$$

где $\widehat{H}_0 = \frac{\hbar\omega_0}{2}\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix} = \frac{\hbar\omega_0}{2}\sigma_z$ – невозмущенный гамильтониан,

$\hat{d} = e\hat{r} = \begin{bmatrix}0 & d_{12}\\ d_{21} & 0\end{bmatrix}$ – оператор дипольного момента

Оператор дипольного момента имеет отличные от нуля элементы только для переходов между состояниями, волновые функции которых обладают различной четностью. Поэтому, во всяком случае, равны нулю все диагональные элементы. Другими словами, равны нулю средние значения

дипольного момента любой системы частиц (например, атома) в стационарных состояниях.[3]

Пусть вектор электрического поля $\vec{E}$ коллинеарен вектору дипольного момента и его напряженность изменяется по закону $E = \cos(\omega t + \varphi)$, в таком случае можно переписать гамильтониан через матрицы Паули:

$$\widehat{H} = \frac{\hbar\omega_0}{2}\sigma_z - d_{21}E_0\cos(\omega t + \varphi)\sigma_x \quad (10)$$

Для рассмотрения гамильтониана с осцилляциями электрического поля частотой ω, удобно сделать переход во вращающуюся рамку, которая вращается с той же частотой. [4] Это означает, что каждому из базисных векторов мы добавляем зависящий от времени фазовый множитель, переходя таким образом во вращающуюся систему координат в пространстве состояний. Такой переход описывается унитарным оператором:

$$|\psi'(t)\rangle = U|\psi(t)\rangle = e^{i\sigma_z t\frac{\omega}{2}}|\psi(t)\rangle \quad (11)$$

Базисные векторы:

$$|0\rangle \to e^{i\omega t/2}|0\rangle;\ \ |1\rangle \to e^{-i\omega t/2}|1\rangle \quad (12)$$

Новый гамильтониан в таком случае:

$$\widehat{H} \to U\widehat{H}U^{\dagger} + i\hbar\frac{\partial}{\partial \mathrm{t}}UU^{\dagger} = \widehat{H}' \quad (13)$$

$$\widehat{H}' = \frac{\hbar\,(\omega_0 - \omega)}{2}\sigma_z + \frac{d_{12}E_0}{2}((\cos(2\omega t + \varphi)\sigma_x - \sin(2\omega t + \varphi)\sigma_y) + (\cos(\varphi)\sigma_x + \sin(\varphi)\sigma_y))$$

Так как эволюция суперпозиции (1) собственных функций невозбужденного гамильтониана в соответствии с уравнением Шредингера происходит следующим образом:

$$|\psi(t)\rangle = \alpha e^{-it\frac{\omega_0}{2}}|0\rangle + \beta e^{it\frac{\omega_0}{2}}|1\rangle \quad (14)$$

$$\varphi(t) = \omega_0 t$$

где $\varphi(t)$ – относительная фаза между $|0\rangle$ и $|1\rangle$.

Относительная фаза проходит интервал от 0 до 2π за промежуток времени $\frac{2\pi}{\omega_0}$, это соответствует вращению вектора состояния кубита вокруг оси Z на сфере блоха в изначальной рамке с частотой ω_0.

Частота $\Delta = (\omega_0 - \omega)$ – называется детюнингом (расстройкой), это скорость остаточной прецессии вокруг оси Z во вращающейся рамке. [5]

Далее можно использовать RWA - приближение вращающейся волны. Ее идея заключается в том, что члены, осциллирующие с частотой $2\omega t$, меняют свой знак много раз за характерное время динамики кубита и их вклад в эволюцию устремляется к нулю. Это условие выполняется, когда $2\omega \gg \frac{d_{12}E_0}{\hbar}, (\omega_0 - \omega)$. Величина $\Omega = \frac{d_{12}E_0}{\hbar}$ - называется Раби частотой.

Итоговый гамильтониан:

$$\widehat{H} = \frac{\hbar\,\Delta}{2}\sigma_z + \frac{\hbar\Omega}{2}(\cos(\varphi)\sigma_x + \sin(\varphi)\sigma_y) \tag{15}$$

Этот гамильтониан можно представить следующим образом:

$$\widehat{H} = \frac{\hbar\Omega_{eff}}{2}(\vec{n}\cdot\vec{\sigma}) \tag{16}$$

где $\vec{n} = \frac{1}{\Omega_{eff}}(\Omega\cos(\varphi),\ \Omega\,sin(\varphi),\ \Delta)$

Здесь появляется множитель Ω_{eff}, необходимый для выполнения равенства $|\vec{n}|^2 = 1$. Он задается уравнением:

$$\Omega_{eff} = \sqrt{\Omega^2 + \Delta^2} \tag{17}$$

В то время как Раби частота Ω $(\frac{\text{рад}}{\text{с}})$ является угловой скоростью вращения вокруг вектора $\vec{n}_{XY} = (\Omega\cos(\varphi),\ \Omega\sin(\varphi), 0)$, величина Ω_{eff}, её называют эффективной Раби частотой, это полная угловая скорость вектора состояния, учитывающая вращения как от Ω, так от детюнинга Δ. Компонента вращения связанная с детюнингом по сути является фиктивной и появляется

из-за соответствующего выбора системы отсчета, однако тем не менее активно используется в квантовом управлении.[6] Легко видеть, что при совпадении частоты электромагнитного импульса и собственной частоты системы ω_0, $\Omega = \Omega_{eff}$, такой режим называется резонансным и соответствует вращению вокруг вектора $\vec{n}_{XY}$, лежащего в плоскости XY.

Если перейти в систему единиц измерения, где $\hbar \equiv 1$. То эволюция системы опишется следующим образом:

$$|\psi(t)\rangle = e^{-i\frac{\theta}{2}(\vec{\sigma}\cdot\vec{n})}|\psi(0)\rangle \quad (18)$$

где $\theta = \Omega_{eff}t$

Экспонента в (18) в точности повторяет выражение (3). Таким образом изменяя Раби частоту Ω (а значит амплитуду электрического поля, в случае если $E(t) = const$), частоту электрического поля ω и начальную фазу импульса φ, можно получить любой унитарный оператор.

Симуляция Раби импульсов

В идеализированной системе, описываемой гамильтонианом вида (15) параметры импульса Раби, который реализует заданное унитарное преобразование, можно легко извлечь из матрицы унитарного оператора. Чтобы понять, как это можно осуществить, рассмотрим произвольный унитарный оператор 2×2, его матрицу можно представить как:

$$U = e^{-i\gamma} e^{-i\frac{\theta}{2}(\vec{\sigma}\cdot\vec{n})} = e^{-i\gamma}\left(I cos\frac{\theta}{2} - i\,(\vec{\sigma}\cdot\vec{n}) sin\frac{\theta}{2}\right) \tag{19}$$

Он соответствует повороту сферы блоха на положительно определенный угол θ вокруг оси $\vec{n}$ против часовой стрелки. Для начала избавимся от фазового множителя $e^{-i\gamma}$, чтобы получить унитарную матрицу с определителем равным 1:

$$\widetilde{U} = \frac{U}{det(U)} \tag{20}$$

$$\widetilde{U} = \begin{bmatrix} \widetilde{U}_{00} & \widetilde{U}_{01} \\ \widetilde{U}_{10} & \widetilde{U}_{11} \end{bmatrix} = \begin{bmatrix} cos\frac{\theta}{2} - in_z sin\frac{\theta}{2} & -n_y sin\frac{\theta}{2} - in_x sin\frac{\theta}{2} \\ n_y sin\frac{\theta}{2} - in_x sin\frac{\theta}{2} & cos\frac{\theta}{2} + in_z sin\frac{\theta}{2} \end{bmatrix} \tag{21}$$

Из матрицы можно легко получить θ:

$$\theta = 2\cdot cos^{-1}\left(\frac{\widetilde{U}_{00}+\widetilde{U}_{11}}{2}\right) \tag{22}$$

И компоненты $\vec{n}$:

$$n_z = Im\left(\frac{\widetilde{U}_{11}-\widetilde{U}_{00}}{2\cdot sin\frac{\theta}{2}}\right) \tag{23}$$

$$n_y = Re\left(\frac{\widetilde{U}_{10}-\widetilde{U}_{01}}{2\cdot sin\frac{\theta}{2}}\right)$$

$$n_x = Im\left(\frac{\widetilde{U}_{01}-\widetilde{U}_{10}}{2\cdot sin\frac{\theta}{2}}\right)$$

Вспоминая выражение для $\vec{n}$:

$$\vec{n} = \frac{1}{\Omega_{eff}}(\Omega\cos(\varphi), \Omega\, sin(\varphi), \Delta) \tag{24}$$

Если взять за основу прямоугольный Раби импульс ($\Omega_{eff}(t) = const$), длинной $t = 1$ (характерный период, за который система с $\Omega_{eff} = 2\pi\left(\frac{\text{рад}}{\text{ед.времени}}\right)$ проходит полный круг и возвращается в изначальное положение) , то из θ и $\vec{n}$ можно получить его параметры следующим образом:

$$\theta = t\Omega_{eff} = \Omega_{eff} \tag{25}$$

$$\omega = \omega_0 + \Delta = \omega_0 + n_z\Omega_{eff}$$

$$\varphi = \begin{cases} tan^{-1}\left(\frac{n_y}{n_x}\right), n_x > 0 \\ tan^{-1}\left(\frac{n_y}{n_x}\right) + \pi, n_x < 0, n_y \geq 0 \\ tan^{-1}\left(\frac{n_y}{n_x}\right) - \pi, n_x < 0, n_y < 0 \end{cases}$$

Формула для фазы является формулой перехода от декартовых координат $\{n_x, n_y\}$ к полярным.

Далее необходимо определить Раби частоту $\Omega = d_{12}E_0$, это легко делается из соотношения:

$$\Omega_{eff}^2 = \Omega^2 + \Delta^2 \tag{26}$$

$$\left(\frac{\theta}{t}\right)^2 = \Omega^2 + \left(n_z\frac{\theta}{t}\right)^2$$

$$\Omega = \sqrt{1-n_z}\frac{\theta}{t} = \sqrt{1-n_z}\theta\ \left(\frac{\text{рад}}{\text{с}}\right)$$

Таким образом, анализ матрицы оператора унитарной эволюции идеализированной двухуровневой системы позволяет определить соответствующий ей импульс Раби. Для простоты и однозначности можно

использовать прямоугольную форму импульса с фиксированной длительностью.

В рамках данной работы, с использованием библиотеки Qiskit Dynamics, разработанной компанией IBM, для языка python, производились симуляции подачи на кубит различных прямоугольных Раби импульсов, для реализации соответствующих им унитарных операторов. Численные значения физических величин использовавшихся в симуляции не были призваны повторять какие либо реальные системы, но соотношения между ними были выбраны таким образом, чтобы работали все описанные выше приближения.

Например, на рисунке 2 изображено действие на состояние кубита вентиля Адамара, его матрица в форме с определителем равным 1, имеет следующий вид:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix} i & i \\ i & -i \end{bmatrix} \tag{27}$$

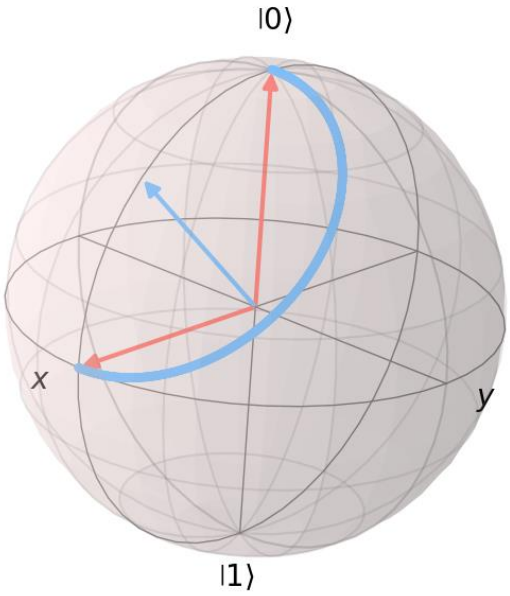


Рисунок 2. Действие Вентиля Адамара на состояние $|0\rangle$, переводящее его в состояние $|+\rangle$

Синей стрелкой показана ось вращения, вокруг которой происходит изменение состояния, а красными – начальное и конечное состояния. Для

численного моделирования временной интервал от 0 до 1 был разбит на 1000 точек, которые на сфере Блоха образуют сплошную линию, отражающую траекторию изменения состояния. Рисунок был простроен с помощью библиотеки QuTiP.

Так же можно рассмотреть вентиль, поворачивающий состояние на угол $\frac{\pi}{2}$ вокруг оси Y, его матрица:

$$Y\left(\frac{\pi}{2}\right) = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \tag{28}$$

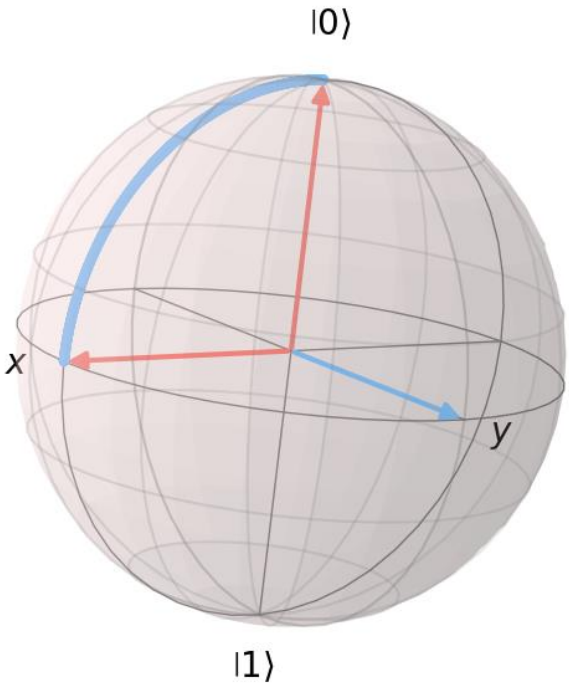


Рисунок 2. Действие Вентиля $Y\left(\frac{\pi}{2}\right)$ на состояние $|0\rangle$

Можно видеть, что хотя конечное состояние в результате действия вентилей одинаково, они соответствуют различным физическим процессам и различным унитарным преобразованиям.

Как последний пример, можно привести симуляцию случайно сгенерированного вентиля, имеющего унитарную матрицу:

$$U = \begin{bmatrix} 0{,}4235 - 0.6245i & -0.0332 + 0.6553i \\ 0.0332 + 0.6553i & 0{,}4235 - 0.6245i \end{bmatrix} \tag{29}$$

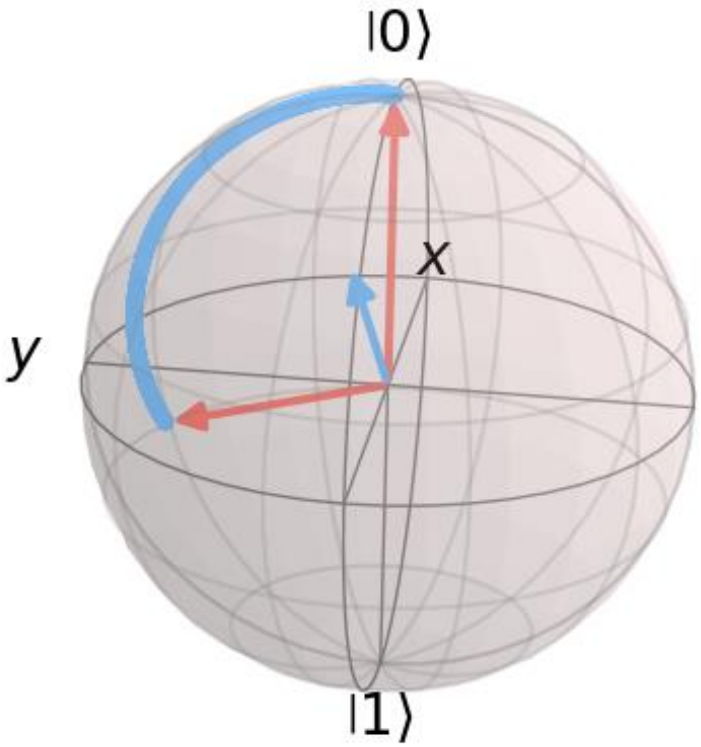


Рисунок 3. Действие Вентиля U на состояние $|0\rangle$

Обучение нейронной сети

Целью этой части работы является обучение нейросети, способной сопоставлять любой унитарной матрице с единичным определителем соответствующий прямоугольный Раби импульс. Для этого она должна правильно подбирать фазу, частоту и амплитуду импульса. Подавать унитарный оператор на вход нейросети предполагается в виде коэффициентов разложения в базисе Паули:

$$U = a_0 I - i a_x \sigma_z - i a_y \sigma_z - i a_z \sigma_z \quad (30)$$

Это позволяет подавать на вход 4 не комплексных, а действительных числа: a_0, a_x, a_y, a_z. Нейронная сеть таким образом должна научиться аппроксимировать отображение $\mathbb{R}^4 \to \mathbb{R}^3$. Качество аппроксимации оценивается функцией потерь MSE, которая измеряет среднеквадратичное отклонение предсказаний сети от истинных значений.

$$MSE = \frac{1}{N} \sum_{l=1}^{N} \|\widehat{y_l} - y_l\|^2 \quad (31)$$

где $\widehat{y_l} \in \mathbb{R}^3$ – выход нейросети

$\widehat{y_l} \in \mathbb{R}^3$ – истинный ответ

N – количество элементов в батче, усреднение по батчу необходимо для получения более плавных градиентов

Во время обучения вычисляется градиент этой ошибки по всем весам сети с помощью обратного распространения ошибки, после чего оптимизатор Adam обновляет параметры, адаптивно подбирая шаг обучения для каждого веса на основе скользящих средних градиентов и их квадратов.

Нейронная сеть имеет 4 полносвязных слоя со следующим количеством нейронов:

$$4 \to 64 \to 128 \to 64 \to 3 \quad (32)$$

В качестве функции активации после каждого слоя идет гиперболический тангенс, который добавляет сети нелинейность.

Генерация датасета, большого набора отображений унитарный оператор → параметры Раби импульса, происходит с использованием формул (21) – (26), скрипт в Приложении 3. Далее происходит процесс обучения нейросети.

Как показатель точности предсказаний нейросети было предложено использовать среднеквадратичное отклонение предсказанного нейросетью вектора блоха от вычисленного аналитически:

$$\sigma = \sqrt{(\Delta x)^2+(\Delta y)^2 + (\Delta z)^2} \tag{33}$$

В результате многих миллионов итераций обучения удалось добиться $\sigma \approx 0.008 \approx 0.8\%$. Что в целом, является неплохим показателем.

Далее представлены визуализации предсказаний нейросети. Зеленым вектором обозначен результат вычисленный аналитически, красными – начальное и конечное состояние системы, претерпевающей эволюцию под воздействием Раби импульса предсказанного нейросетью. На рисунке 4 изображен вентиль $Y\left(\frac{\pi}{2}\right)$. $\sigma \approx 0.019$. Можно визуально заметить несовпадение

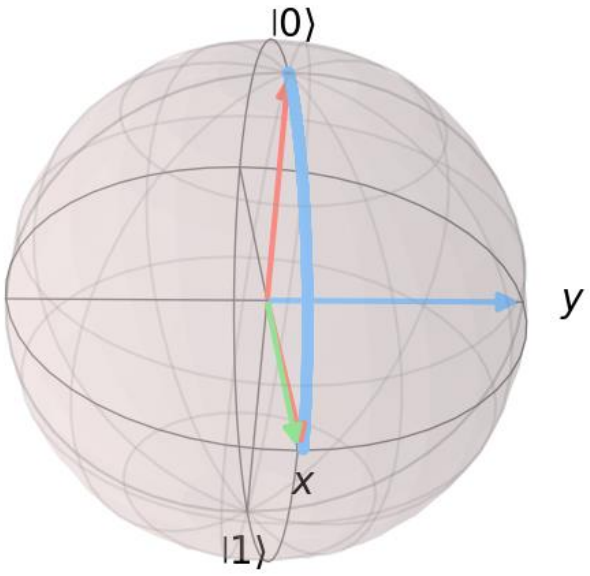


Рисунок 4. Предсказанный нейросетью вентиль $Y\left(\frac{\pi}{2}\right)$

На следующем рисунке показаны 2 произвольных вентиля предсказанных нейросетью. Слева $\sigma \approx 0.009$, справа $\sigma \approx 0.006$ (аналитический и предсказанный нейросетью вектора практически совпадают).

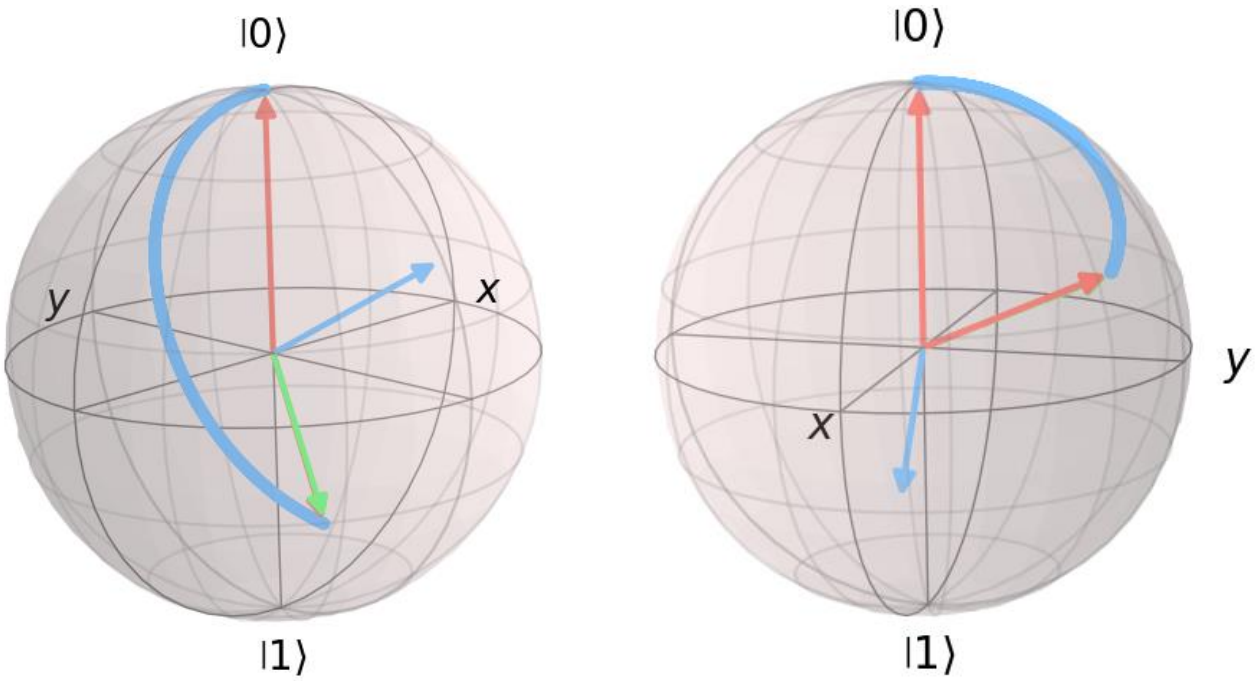


Рисунок 5. Предсказанные нейросетью произвольные унитарные преобразования

Заключение

В ходе работы были изучены важные элементы квантового управления, такие как кубиты, квантовые вентили и Раби осцилляции, а также некоторые методы рассмотрения квантовых систем, такие как переход во вращающуюся рамку, дипольное приближение, дипольная калибровка электромагнитного потенциала, приближение вращающейся волны и различные представления унитарных операторов, играющих ключевую роль во всей квантовой механике.

Были проведены физические симуляции эволюции идеализированного кубита под действием прямоугольных Раби импульсов. На основе эффекта Раби осцилляций продемонстрировано формирование квантовых вентилей, применяемых на практике.

На финальном этапе работы была обучена нейросеть, призванная подбирать правильные параметры прямоугольного Раби импульса для любых унитарных операторов. Среднеквадратичное отклонение предсказанного нейросетью вектора блоха от вычисленного аналитически $\sigma \approx 0.008$. Что является достаточно неплохим результатом. Однако, весьма вероятно, что в будущем удастся добиться более эффективного и точного обучения нейронной сети, так как текущий результат не является оптимальным. Хотя для абсолютного большинства произвольных унитарных матриц нейросеть дает достаточно точные прогнозы, по еще не объясненным причинам в случае некоторых унитарных операторов результаты работы нейросети остаются неудовлетворительными.

Также стоит отметить, что задача, рассмотренная в данной работе, является очень значительным упрощением, из-за отсутствия диссипации, шумов и прочих паразитных эффектов, рассматриваемая система остается далека от реальных физических реализаций. Во многом благодаря этому параметры Раби импульсов для такой системы намного проще рассчитывать аналитически.

Список литературы

[1] М. Нильсен., И.Чанг «Квантовые вычисления и квантовая информация»: Пер. с англ. - М.: Мир, 2006 г.

[2]КВАНТОВАЯ МЕХАНИКА, Лекция №12: Движение частицы в электромагнитных полях. Статическое магнитное поле А.Г. Семенов

[3]Л а н д а у Л. Д., Л и ф ш и ц Е. М. Теоретическая физика: Учеб. пособ.: Для вузов. В 10 т. Т. III. Квантовая механика (нерелятивистская теория) / Под ред. Л.П. Питаевского. — 6-е изд., испр. — М.: ФИЗМАТЛИТ, 2021. — 800 с.

[4]https://qopt.readthedocs.io/en/latest/examples/rabi_rwa.html

[5]Introduction to Quantum Computing, Ray LaPierre. The Materials Research Society Series 2021

[6]Observing hyperfine interactions of NV− centers in diamond in an advanced quantum teaching lab

Приложение

1. Код скрипта SchrodingerEquation.py В нем происходит генерация случайных унитарных операторов, расчет и физическая симуляция Раби импульса.

```
from pygments.styles.gh_dark import BLUE_2, RED_3, RED_2, GREEN_1
from qiskit_dynamics.solvers import Solver
from qiskit_dynamics.signals import Signal
from qiskit.quantum_info import Pauli
from qiskit.quantum_info.states import Statevector
import matplotlib.pyplot as plt
import random
import qutip
import numpy as np


X = Pauli("X").to_matrix()
Y = Pauli("Y").to_matrix()
Z = Pauli("Z").to_matrix()


t_ev = np.linspace(0., 1, 1000)
Freq_0 = 200.0 # Собственная частота кубита, в неких условных единицах


Y_POL_PI = np.array([[1/ np.sqrt(2), -1/ np.sqrt(2)],
           [1/ np.sqrt(2), 1/ np.sqrt(2)]])

HADAMAR = np.array([[-1j / np.sqrt(2), -1j / np.sqrt(2)],
           [-1j / np.sqrt(2), 1j / np.sqrt(2)]])


def make_Rabi_impulse(Amp, sigma, T):
    def Rabi_impulse(t):
        global sumd
        return np.where(t < T,
                        Amp,
                        0.0)
    return Rabi_impulse

def f(x):
    return 1


#Создаем случайную унитарную матрицу
def Make_random_Unitar():
    norm = np.random.randn(3)
    norm /= np.linalg.norm(norm)
    theta = np.random.uniform(0, 2 * np.pi)
    Unitar = np.zeros((2,2), dtype = complex)
    Unitar[0][0] = np.cos(theta / 2) - 1j * norm[2] * np.sin(theta / 2)
    Unitar[1][1] = np.cos(theta / 2) + 1j * norm[2] * np.sin(theta / 2)
    Unitar[0][1] = - np.sin(theta / 2) * norm[1] - 1j * norm[0] *
np.sin(theta / 2)
    Unitar[1][0] = np.sin(theta / 2) * norm[1] - 1j * norm[0] * np.sin(theta
/ 2)

    return Unitar, norm

#Делаем разложение по базису Паули унитарной матрицы
```

```
#Чтобы в таком виде передавать в нейросеть
def Pauli_from_Unitar(Unitar):
    global X
    global Y
    global Z
    a0 = 0.5 * np.trace(Unitar)
    ax = 0.5 * np.trace(Unitar @ X)
    ay = 0.5 * np.trace(Unitar @ Y)
    az = 0.5 * np.trace(Unitar @ Z)
    return [a0.real, ax.imag, ay.imag, az.imag]

# Тут мы считаем конечное ожидаемое состояние
#просто умножая вектор на унитар
# можно сделать случайное начальное состояние
def Generate_random_state(gate, alpha = complex(random.random(),
random.random()),
                          beta = complex(random.random(), random.random())):
    module_alpha = np.abs(alpha)
    module_beta = np.abs(beta)
    alpha = alpha / (np.sqrt(module_alpha ** 2 + module_beta ** 2))
    beta = beta / (np.sqrt(module_alpha ** 2 + module_beta ** 2))
    y_rand = np.array([alpha, beta])
    sv_init = Statevector(y_rand)
    y_rand = gate @ y_rand
    sv_final = Statevector(y_rand)
    bloch_sv_init = np.array([sv_init.expectation_value(X).real,
sv_init.expectation_value(Y).real, sv_init.expectation_value(Z).real])
    bloch_sv_final = np.array([sv_final.expectation_value(X).real,
sv_final.expectation_value(Y).real, sv_final.expectation_value(Z).real])
    return sv_init, sv_final, bloch_sv_final

#Получаем из матрицы параметры прямоугольного Раби импульса
def Calculate_impulse(gate):
    Gate = gate
    angle = 2 * np.arccos((Gate[1][1] + Gate[0][0]) / 2)
    norm = [ - (Gate[1][0] + Gate[0][1]) / (2 * np.sin(angle / 2)),
(Gate[1][0] - Gate[0][1]) / (2 * np.sin(angle / 2)),
        - (Gate[0][0] - Gate [1][1]) / (2 * np.sin(angle / 2))]
    norm = np.array([norm[0].imag, norm[1].real, norm[2].imag])
    Freq_E = Freq_0 - norm[2] * angle / (2 * np.pi)
    phase = np.arctan2(norm[1],norm[0])
    Freq_Rabi = angle / (2 * np.pi) * np.sqrt(1 - (norm[2] ** 2))
    Amp =  1
    return phase, Freq_E, Freq_Rabi, norm

# Физическая симуляция
def Simulate_impulse(sv_init, t_eval, phase, Freq_E, Freq_Rabi):
    signal = Signal(envelope = 1, carrier_freq = Freq_E, phase = phase)
    signal.draw(0, 1, 1000)
    H_0 = 0.5 * 2 * np.pi * Freq_0 * Z
    H_field = 2 * np.pi * Freq_Rabi * X
    evolution = Solver( static_hamiltonian = H_0, hamiltonian_operators =
[H_field], rotating_frame = (Freq_E/200)*H_0)
    result = evolution.solve(t_span = [0., 1], y0 = sv_init, signals =
[signal], t_eval = t_eval,     atol=3e-14,rtol=3e-14)
    res = result
    bloch = np.zeros((len(res.y), 3))
    for i in range (len(res.y)):
        bloch[i] = [res.y[i].expectation_value(X).real,
res.y[i].expectation_value(Y).real, res.y[i].expectation_value(Z).real]
    return bloch

#Рисуем график
```

```
def Visualize (bloch, norm, bloch_sv_final = np.zeros(3)):
    b = qutip.Bloch()
    bloch = bloch.T
    if bloch_sv_final[0] !=0:
        b.add_vectors(bloch_sv_final, colors=GREEN_1)
    print(bloch[:, 1])
    b.add_vectors(bloch[:, 0], colors=RED_3)
    b.add_vectors(bloch[:, 999], colors=RED_3)
    b.add_vectors(norm, colors=BLUE_2)
    b.add_points(bloch, colors=BLUE_2)
    b.show()
    plt.show()

if __name__ == "__main__":

    New_Unitar, norm = Make_random_Unitar()
    New_Unitar = HADAMAR
    print(New_Unitar)
    phase, Freq_E, Freq_Rabi, norm = Calculate_impulse(New_Unitar)
    sv_init, sv_final, bloch_sv_final = Generate_random_state(New_Unitar, 1,
0)
    bloch = Simulate_impulse(sv_init, t_ev, phase, Freq_E, Freq_Rabi)
    Visualize(bloch, norm )
```

2. Код скрипта NeuralNetwork.py в котором задается архитектура и порядок обучения нейросети, а также ее проверка.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.optim.lr_scheduler import ExponentialLR
import SchrodingerEquation as Sch

input1 = np.load("input.npy")
output = np.load("output.npy")
output_mean = output.mean(axis=0)
output_std = output.std(axis=0)
output = (output - output_mean) / output_std
input1 = torch.from_numpy(input1).to(torch.float64)
output = torch.from_numpy(output).to(torch.float64)



t_ev = np.linspace(0., 1, 1000)

# Архитектура нейросети
class QuantumNet(nn.Module):
    def __init__(self):
        super(QuantumNet, self).__init__()
        self.layer1 = nn.Linear(4, 64)
        self.layer2 = nn.Linear(64, 128)
        self.layer3 = nn.Linear(128, 64)
        self.layer4 = nn.Linear(64, 3)

    def forward (self, x):
        x = torch.tanh(self.layer1(x))
        x = torch.tanh(self.layer2(x))
```

```
        x = torch.tanh(self.layer3(x))
        return self.layer4(x)


class MyDataset(Dataset):
    def __init__(self, input, output):
        self.input = input
        self.output = output

    def __len__(self):
        return len(self.input)

    def __getitem__(self, idx):
        x = self.input[idx]
        target = self.output[idx]
        return x, target
dataset = MyDataset(input1, output)
data = DataLoader(
    dataset,
    batch_size=512,
    shuffle=True
)

model = QuantumNet()
model = model.double()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()
model.train()
print(input1)
scheduler = ExponentialLR(optimizer, gamma=0.99999816)
num = int(input())

if num == 1:      #Загружаем из файла
    model.load_state_dict(torch.load("model.pth"))
    model.eval()
else:               # Или обучаем
    for i in range(100):
        for x, target in data:
           optimizer.zero_grad()
           output = model (x)
           loss = criterion(output, target)
           loss.backward()
           optimizer.step()
           scheduler.step()
           print(loss)
    model.eval()
delta =0

for i in range(100):      # Проверка
    New_X, norm = Sch.Make_random_Unitar()
    New_X_formated = Sch.Pauli_from_Unitar(New_X)
    New_X_formated = torch.tensor(New_X_formated, dtype=torch.float64)
    Y_pred = model(New_X_formated)
    Y_pred_new = Y_pred.detach().numpy()
    Y_pred_new = Y_pred_new * output_std + output_mean
    print("Придуманный импульс", Y_pred_new)
    phase, Freq_E, Freq_Rabi, norm = Sch.Calculate_impulse(New_X)
    print("Посчитанный импульс", phase, Freq_E, Freq_Rabi)
    sv_init, sv_final, bloch_sv_final = Sch.Generate_random_state(New_X, 1,
0)
    bloch = Sch.Simulate_impulse(sv_init, t_ev, Y_pred_new[0], Y_pred_new[1],
Y_pred_new[2])
    print("Посчитанный блох", bloch_sv_final)
```

```
    print("Придуманный блох",bloch[999])
    delta += np.sqrt((bloch[999][0]- bloch_sv_final[0]) ** 2 +
(bloch[999][1]- bloch_sv_final[1]) ** 2
                     + (bloch[999][2]- bloch_sv_final[2]) ** 2)
    print("Средняя длина вектора ошибки",delta / (1+i))
    Sch.Visualize(bloch, norm, bloch_sv_final)
print(delta/100)
torch.save(model.state_dict(), "model.pth")
```

3. Код скрипта DatasetGenerator.py в котором происходит генерация массива пар: Унитарный оператор – Раби импульс, нужных для обучения нейронной сети.

```
import numpy as np
import SchrodingerEquation as Sch

input_arr = np.zeros((5000000,4), dtype=complex )
output_arr = np.zeros((5000000,3), dtype=complex)
New_Unitar = Sch.Make_random_Unitar()
for i in range(5000000):
    if i % 100000 == 0:
        print(i)
    New_Unitar, norm = Sch.Make_random_Unitar()
    Unitar_Pauli = Sch.Pauli_from_Unitar(New_Unitar)
    phase, Freq_E, Freq_Rabi, norm  = Sch.Calculate_impulse(New_Unitar)
    Freq_E = Freq_E.real
    Freq_Rabi = Freq_Rabi.real
    output_arr[i] = [phase, Freq_E, Freq_Rabi]
    input_arr[i] = Unitar_Pauli
print (input_arr)

np.save("input.npy", input_arr)
np.save("output.npy", output_arr)
```